

Runtime 规范

定位

Runtime 是千依的核心执行引擎，以 MatterContext 为共享上下文，根据 Flow 节点类型与当前执行状态选择并执行对应指令，将执行结果写回 MatterContext。对上游而言是可靠的、内敛的服务。

可靠性模型

ExecutionContext

```
public class ExecutionContext {  
    private String matterId;  
    private String nodeId;  
    private MatterContext matterContext;  
    private FlowDefinition flowDefinition;  
    private FlowNode currentNode;  
}
```

- 虚拟线程创建时一次性加载，不可变，不提供 setter
- pendingUpdates 提交成功后从 MongoDB 重新读取 MatterContext，替换 ExecutionContext 为最新快照，随即清空 pendingUpdates

pendingUpdates

```
Map<String, Object> pendingUpdates = new LinkedHashMap<>();
```

- key 统一使用 MongoDB 字段路径，例如：`context.start.amount`、`executionStates.start.status`、`executionStates.start.cursor`、`assigneeMapping.node1`
- 副作用触发时整个 Map 直接作为 \$set 参数原子提交，无需任何转换
- 提交成功后立即清空，防止重复提交
- 同一 key 多次写入时 LinkedHashMap 天然保留最新值

context 字段查阅

```
public Object getContextValue(String contextKey) {  
    String mongoKey = "context." + contextKey;  
    if (pendingUpdates.containsKey(mongoKey)) {  
        return pendingUpdates.get(mongoKey);  
    }  
    return executionContext.getMatterContext()  
        .getContext()  
        .get(contextKey);  
}
```

- pendingUpdates 优先，ExecutionContext 兜底
- 使用 containsKey 防御 value 为 null 的合法写入被误判为未命中
- 所有 context 字段读取必须经过此方法，禁止直接访问 pendingUpdates 或 MatterContext.context

执行原则

- 纯内存操作只修改内存，cursor 在内存中推进
- 有副作用的操作：先持久化所有内存修改和 cursor，再执行副作用
- ACK 时机：写入 TRIGGERED 后立即 ACK，RocketMQ 的 at-least-once 覆盖消费到写 TRIGGERED 的窗口，写 TRIGGERED 之后的恢复不依赖 RocketMQ 重投，依赖 cursor 机制由 RecoveryRunner 驱动恢复
- 出度触发：必须经过 RocketMQ 事务消息，禁止直接创建虚拟线程触发子节点
- 持久化在前，投递消息在后，顺序不可颠倒
- ActionChainName 是业务语义标签，标识当前执行所处的业务阶段，不驱动任何切换逻辑，失败处理内联到正常执行流程中

幂等守卫 所有节点执行前必须经过幂等守卫校验，规则见 core.message.execution 包规范。

cursor 机制 cursor 记录当前节点下一个待执行的 Action 索引，每次副作用触发前持久化，崩溃后从 cursor 位置恢复，不重复执行已完成的步骤。

消费模型

- 同一 Matter 的所有消息归属于同一 MessageQueue
- 同一 MessageQueue 在同一时刻只由一个 Runtime 实例消费
- RocketMQ 负载均衡天然保证 Matter 级别的执行隔离
- 多出度子节点消息由同一实例顺序消费，通过虚拟线程并发执行

异常体系

系统异常：代码、配置、基础设施问题，投递 SystemErrorMessage 通知 FDE：

- Roster 解析失败
- Task 资产不存在
- input_mapping 引用的 key 不存在
- MatterContext 组装不完整
- nodeId 不存在

业务异常：业务流程问题，投递 BusinessNotificationMessage 通知业务人员：

- RPC 调用失败重试耗尽
- condition 求值异常

同时通知业务人员和 FDE 的场景：

- RPC 调用失败重试耗尽：下游服务可能存在系统性问题
- 熔断器触发：下游服务健康状况需要 FDE 排查

包结构

```
com.qianyi.runtime
└─ recovery          # 崩溃恢复，见 recovery/CLAUDE.md
```

其余包在逐包设计阶段展开。

技术规范

- 虚拟线程：每个节点执行在独立虚拟线程中运行
- MongoDB：字段级原子写入（\$set），禁止整文档替换
- RocketMQ：at-least-once 消费，事务消息保证多出度投递原子性
- OpenFeign：调用 Roster，声明式接口，契约与 core.model 绑定

规划阶段

只实现已在场景库和设计文档中明确记录的内容，禁止任何未经设计的扩展。

代码注释模版

```
/**
 * ${Description}
 *
 * @author TianJunQi
 * @since ${YEAR}-${MONTH}-${DAY}
 */
```